Analog Modulation

Super-heterodyne Receiver

Youssef Mohammed Ibrahim Sayed

Youssef Hisham Ahmed Heikal

Faculty of Engineering
Department of Electronics and Communications
Course code : EECG232-17
Course name : Introduction to Communication Systems

Analog Modulation

Super-heterodyne Receiver

Youssef Mohammed Ibrahim Sayed
Student ID : 91240892
Section : 4
BN : 34

Youssef Hisham Ahmed Heikal
Student ID : 91240907
Section : 4
BN : 36

Supervisor: Dr. Ahmed Hisham
Professor, Cairo University

Co-supervisor: ENG. Mohammed Khaled
Teacher Assistant, Cairo University

Faculty of Engineering
Department of Electronics and Communications

Course code : EECG232-17
Course name : Introduction to Communication Systems

Abstract

This report details a MATLAB-based simulation of a Frequency Division Multiplexed (FDM) communication system employing DSB-SC modulation and a Superheterodyne Receiver. The methodology involves modulating multiple audio signals with 30 kHz channel spacing starting from a 100 kHz carrier frequency. To ensure signal integrity and Nyquist compliance, input signals are processed through interpolation and summed into a monophonic stream.

The receiver architecture implements a tunable RF bandpass filter for image rejection, a mixer with a high-side local oscillator ($f_{LO} = f_c + f_{IF}$), and a fixed 15 kHz Intermediate Frequency (IF) stage to maintain constant selectivity. Final message recovery is achieved through synchronous baseband detection and low-pass filtering. Experimental results validate the system through spectral analysis and audio fidelity tests, specifically quantifying the performance degradation caused by local oscillator frequency offsets and the critical necessity of the RF stage in suppressing image frequency interference. This work demonstrates the practical application of heterodyning principles in achieving high-selectivity signal recovery within a multiplexed environment.

Contents

1 Introduction **2**
1.1 Motivation 2
1.2 System Architecture 2

2 Signal Pre-processing **3**
2.1 Audio Signal Handling 3
2.2 Stereo to Mono Conversion and Length Padding 3
2.3 Spectrum Analysis 4

3 Transmitter Design (AM Modulator) **6**

4 Receiver Design **8**
4.1 RF BPF 8
4.2 The Mixer and Local Oscillator 9
4.3 IF Stage 10
4.4 Baseband Detection 10

5 Removing the RF BPF (Q4) **12**

6 Adding Frequency offset (Q5) **14**
6.1 Adding 0.1 kHz offset 14
6.2 Adding 1 kHz offset 15

Appendices

A Appendix **18**

1 Introduction

1.1 Motivation

The purpose of this project is to simulate the fundamental components of an analog communication system using MATLAB programming. Specifically, an AM modulator and a corresponding super-heterodyne receiver are simulated using radio-station generated signals to analyze signal recovery in a multiplexed environment.

1.2 System Architecture

The communication system architecture comprises several discrete stages. Initially, the baseband message signal is processed by an AM Modulator; this modulation is essential for Frequency Division Multiplexing (FDM) and to optimize the physical dimensions of the antenna as we learned in the lectures. Following transmission, the signal is captured by a super-heterodyne receiver. The front-end consists of a tunable RF Band-Pass Filter (BPF) responsible for image rejection and selecting the desired channel. The selected signal is then down-converted by a mixer—multiplied with a Local Oscillator (LO) signal—to a fixed Intermediate Frequency (IF). Finally, the signal undergoes baseband detection via synchronous demodulation and low-pass filtering to recover the original message.

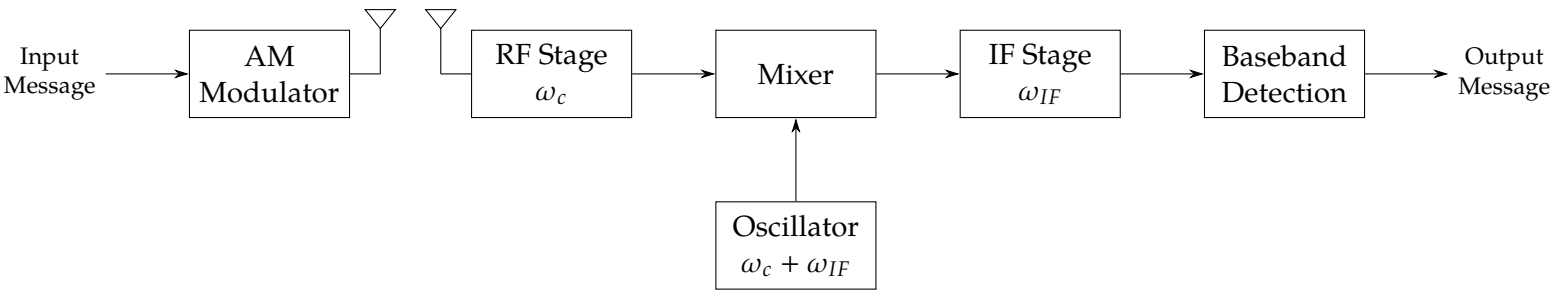

Figure 1.1: *AM modulator and a super-heterodyne receiver.*

2

Signal Pre-processing

2.1 Audio Signal Handling

First, the provided audio files must be read into the workspace. For this implementation, we selected the Quran and BBC radio channels. The primary objective of this stage is to import the audio data and extract the sampling frequency (F_S) for use in subsequent signal processing steps. We can read them in MATLAB using the built-in function audioread, which extracts the discrete-time audio signal along with its sampling frequency.

Listing 1: *Reading the audio files in MATLAB*

```
%% Reading the files
[signal1_raw, Fs1] = audioread('C:/Users/bobyo/Downloads/Short_QuranPalestine.wav');
[signal2_raw, Fs2] = audioread('C:/Users/bobyo/Downloads/Short_BBCArabic2.wav');
```

2.2 Stereo to Mono Conversion and Length Padding

After reading the audio files, we observe that the provided baseband signals are in stereo, meaning each message consists of two channels. Because the objective of this project is to implement a monophonic super-heterodyne receiver, there is no need to process two separate channels. Additionally, we must account for the fact that the given audio signals are not of equal length. This variation poses a challenge for later stages, as multiplexing multiple signals together requires them to be synchronized in size.

We can efficiently resolve both of these structural issues using the following pre-processing methods:

- **Stereo to Mono Conversion:** We can form a single-channel stream by simply adding the two channels together. This combines the audio information into a mono format suitable for our receiver design.
- **Equalizing Lengths:** We find the length of the longest audio file, and pad the short signals with zeros so they all have the exact same length. This zero-padding

technique effectively adds silence to the end of the shorter tracks, ensuring uniform vector lengths in MATLAB without altering the original frequency content of the messages.

Listing 2: *Converting the stereo signals into mono signals and padding the lengths*

```
%% Convert stereo to mono
if size(signal1_raw, 2) == 2, signal1 = signal1_raw(:,1) + signal1_raw(:,2); else, signal1 = signal1_raw; end
if size(signal2_raw, 2) == 2, signal2 = signal2_raw(:,1) + signal2_raw(:,2); else, signal2 = signal2_raw; end

disp(['Fs1 = ', num2str(Fs1), ' Hz']);
disp(['Fs2 = ', num2str(Fs2), ' Hz']);

%% Pad to equal length
len1 = length(signal1);
len2 = length(signal2);
max_len = max(len1, len2);

if len1 < max_len, signal1 = [signal1; zeros(max_len - len1, 1)]; end
if len2 < max_len, signal2 = [signal2; zeros(max_len - len2, 1)]; end
```

2.3 Spectrum Analysis

Following the preprocessing stage, it is essential to analyze the frequency content of the baseband signals to estimate their effective bandwidth. By applying the Fast Fourier Transform (FFT) using MATLAB's `fft` and `fftshift` commands, we can visualize the frequency spectrum of the audio signals. Identifying the maximum frequency component (F_m) present in the messages is a critical prerequisite for the subsequent modulation and receiver design stages. Specifically, knowing the exact signal bandwidth ensures that the frequency-division multiplexed (FDM) channels are spaced appropriately to avoid overlapping, and it provides the necessary cut-off frequency parameters required for designing the system's Low-Pass and Band-Pass filters.

Listing 3: *Converting the stereo signals into mono signals and padding the lengths*

```
%% ========== PLOT SPECTRUM (Original Baseband) ============
N_plot1 = length(signal1);
Y1_shifted = fftshift(fft(signal1));
Y2_shifted = fftshift(fft(signal2));

magnitude1_shifted = abs(Y1_shifted);
magnitude2_shifted = abs(Y2_shifted);
freq_shifted = (-N_plot1/2 : N_plot1/2 - 1) * (Fs1 / N_plot1);

figure('Name', 'Original Baseband Spectra');
```

```
subplot(2,1,1);
plot(freq_shifted / 1000, magnitude1_shifted);
xlabel('Frequency (kHz)'); ylabel('Magnitude'); title('Spectrum of Signal 1 (Quran
↪  Channel)'); grid on; xlim([-25, 25]);

subplot(2,1,2);
plot(freq_shifted / 1000, magnitude2_shifted);
xlabel('Frequency (kHz)'); ylabel('Magnitude'); title('Spectrum of Signal 2 (BBC
↪  Channel)'); grid on; xlim([-25, 25]);
drawnow;

%% Upsampling
upsample_factor = 10;
signal1 = interp(signal1, upsample_factor);
signal2 = interp(signal2, upsample_factor);

original_Fs = Fs1;
Fs1 = Fs1 * upsample_factor;
Fs2 = Fs2 * upsample_factor;
disp(['New Fs1 = ', num2str(Fs1), ' Hz']);
disp(['New Fs2 = ', num2str(Fs2), ' Hz']);
```

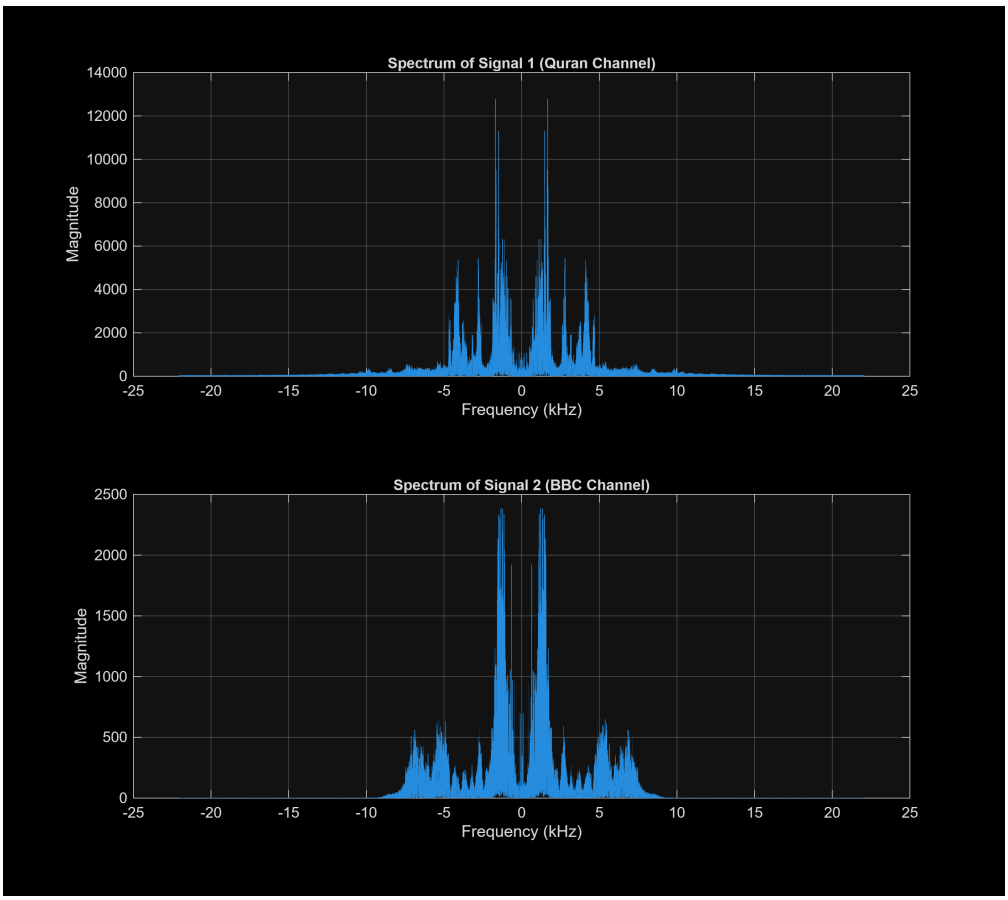

Figure 2.1: *Original Baseband Spectra.*

3

Transmitter Design (AM Modulator)

After extracting the signal and converting it to a monophonic format, we proceed to the next stage: passing the signal to the transmitter block. However, before transmission, we must determine the most suitable modulation scheme for our scenario in terms of cost and implementation complexity. For this specific project, Amplitude Modulation (AM) was chosen.

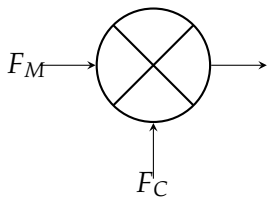

Figure 3.1: *The AM-DSB-SC Modulator (Mixer)*

For Amplitude Modulation - Double sided band - Suppressed carrier (AM-DSB-SC), we need to specify the carrier frequency (F_c) to mix with our message signal, as specfied in our project we will a default $F_c = 100$ kHz and for each n signal it will have $F_n = 100$ kHz $+ n\Delta F$, where $\Delta F = 30$ kHz. After knowing the parameters we need for our mixer we can now go and define them in MATLAB.

Listing 4: *Defining the carrier frequency in MATLAB*

```
%% MODULATION
N1 = length(signal1);
Ts = 1/Fs1;
t = (0:N1-1)' * Ts;

fc = 100e3; % Base carrier frequency
delta_f = 30e3; % Frequency spacing
fc1 = fc;             % Carrier for first signal (100 kHz)
fc2 = fc + delta_f;   % Carrier for second signal (130 kHz)
```

Then now we are ready to pass the signals into the modulator with each signal having its F_c to be mixed with for it to be ready to be sent to the receiver.

Listing 5: *Multiplying each signal with its carrier*

```
Quran_Channel_Modulated = signal1 .* cos(2 * pi * fc1 * t);
BBC_Channel_Modulated = signal2 .* cos(2 * pi * fc2 * t);

FDM = Quran_Channel_Modulated + BBC_Channel_Modulated;
```

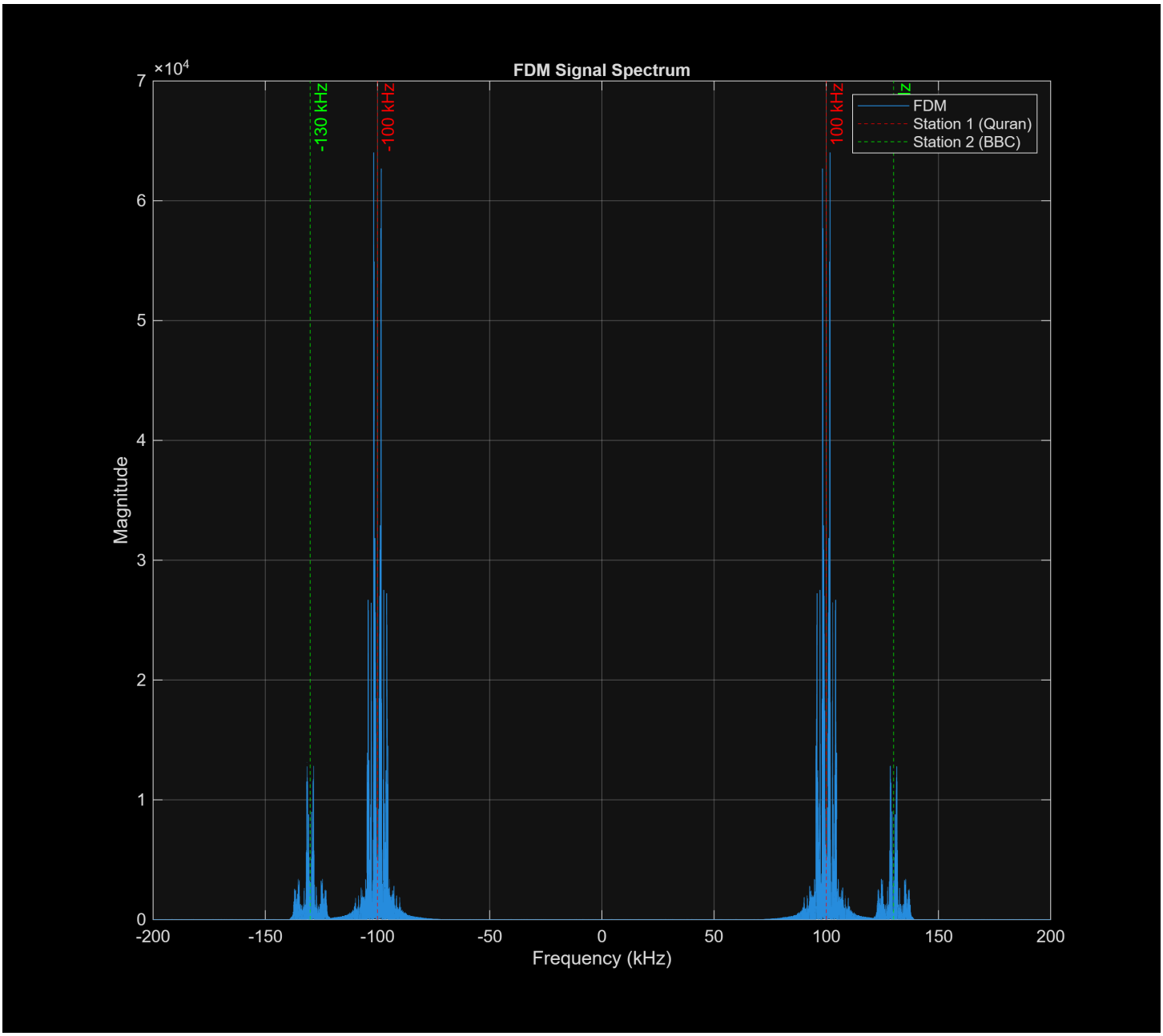

Figure 3.2: *The FDM Spectra.*

4

Receiver Design

Now that the signal is sent from the transmitter, the receiver is ready to receive it, There are various types of receivers but in our project we are going to simulate the super heterodyne receiver.

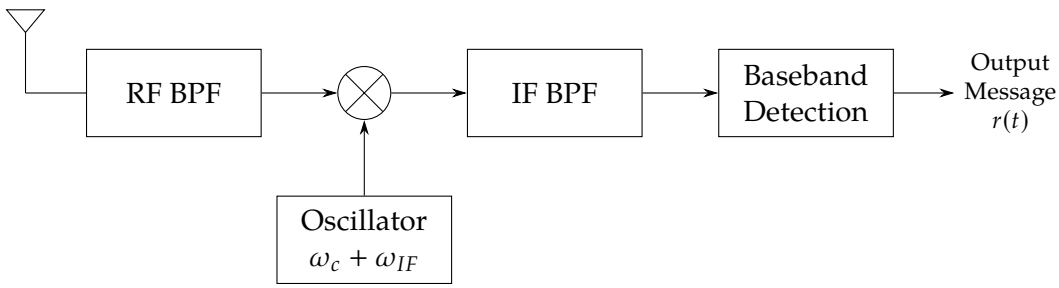

Figure 4.1: *Super-heterodyne receiver block diagram.*

4.1 RF BPF

First, the Radio Frequency Bandpass Filter (RF BPF) is designed to isolate the target signal. Its center frequency (f_c) and bandwidth (BW) are defined according to the specifications of the chosen signal. Since both channels have a $BW = 30$kHz, we will design the filter with that BW, for the Quran channel we will center it at $f_c = 100$kHz and the BBC channel at $f_c = 130$kHz . To implement this filter in MATLAB, the `fdesign` function is utilized. This function requires the specification of the lower and upper stopband and passband frequencies. Additionally, it allows for the selection of the filter order, which is critical for achieving the necessary roll-off and attenuation to ensure effective signal isolation.

Note

The **filter order** defines the steepness of the transition between the passband and stopband. A higher order creates a sharper "brick-wall" effect, allowing for precise signal isolation and better rejection of adjacent interference within the `fdesign` framework.

Listing 6: *Designing the RF BPF parameters in MATLAB*

```
%% Channel Specific RF Filters & Oscillators
RF_Filter_Order = 26;

% Quran Channel RF Filter (100 kHz Center)
RF_PassBand_Low_Quran = 85e3;
RF_PassBand_High_Quran = 115e3;
RF_Filter_Specs_Quran = fdesign.bandpass('N,F3dB1,F3dB2', RF_Filter_Order,
↪  RF_PassBand_Low_Quran, RF_PassBand_High_Quran, Fs1);
RF_filter_Quran = design(RF_Filter_Specs_Quran, 'butter');
RF_output_Quran = filter(RF_filter_Quran, FDM);
F_osc_Quran = fc1 + F_IF;

% BBC Channel RF Filter (130 kHz Center)
RF_PassBand_Low_BBC = 115e3;
RF_PassBand_High_BBC = 145e3;
RF_Filter_Specs_BBC = fdesign.bandpass('N,F3dB1,F3dB2', RF_Filter_Order,
↪  RF_PassBand_Low_BBC, RF_PassBand_High_BBC, Fs1);
RF_filter_BBC = design(RF_Filter_Specs_BBC, 'butter');
RF_output_BBC = filter(RF_filter_BBC, FDM);
F_osc_BBC = fc2 + F_IF;
```

4.2 The Mixer and Local Oscillator

After the RF BPF stage, the signal is translated to the Intermediate Frequency (IF) range. This is achieved by using a **Mixer** to multiply the RF signal with a local carrier produced by the **Local Oscillator (LO)**, shifting the signal to the desired frequency f_{IF}, where $f_{LO} = f_c + f_{IF}$.

Listing 7: *Passing the output of the RF stage to the mixer then to the IF stage*

```
% Mixer Stage
Mixer_Output = current_RF_in .* cos(2 * pi * current_F_osc * t);

% IF Filtering Process
IF_Channel = filter(IF_filter, Mixer_Output);
```

4.3 IF Stage

After translating the signal into the IF range, we pass it through an IF-BPF to refine the signal's shape. At this stage, the filter acts as a precision tool to isolate the desired message from the various frequencies produced during mixing. This ensures that only the intended signal remains, effectively clearing away any surrounding interference before the final detection.

Listing 8: *Designing the IF BPF parameters in MATLAB*

```
F_IF = 15e3;

% IF Filtering Specs
IF_Filter_Order = 8;
IF_PassBand_Low = 5e3;
IF_PassBand_High = 30e3;
IF_Filter_Specs = fdesign.bandpass('N,F3dB1,F3dB2', IF_Filter_Order, IF_PassBand_Low,
↪  IF_PassBand_High, Fs1);
IF_filter = design(IF_Filter_Specs, 'butter');
```

4.4 Baseband Detection

In the final stage of the receiver, the signal is translated from the Intermediate Frequency (IF) down to the baseband. To achieve this, the IF signal is first passed through a **Mixer**, which shifts the spectrum down to the zero-frequency range. Subsequently, the output is routed through a **Low-Pass Filter (LPF)**. The LPF is then used to reject high-frequency mixing products and extract the clean message.

Listing 9: *Designing the LPF parameters in MATLAB*

```
% LPF Specs
LPF_Filter_Order = 8;
LPF_Cutoff = 10e3; % 10 kHz cutoff
LPF_Specs = fdesign.lowpass('N,F3dB', LPF_Filter_Order, LPF_Cutoff, Fs1);
LPF_filter = design(LPF_Specs, 'butter');
```

Listing 10: *Returning the signal to Baseband*

```
% Return to Baseband & Low-Pass Filtering
baseband_mixed = IF_Channel .* cos(2 * pi * F_IF * t);
demodulated = filter(LPF_filter, baseband_mixed);
demodulated = demodulated - mean(demodulated); % remove dc offset

% Downsample back & Normalize
demod_down = downsample(demodulated, upsample_factor);
demod_down = demod_down / max(abs(demod_down));
```

Listing 11: *Passing each channel parameters to the receiver*

```
%% DEMODULATION

% Quran
current_RF_in = RF_output_Quran;
current_F_osc = F_osc_Quran;
station_name = 'Quran Channel';
current_condition = 'Ideal';
receiver;

% BBC
current_RF_in = RF_output_BBC;
current_F_osc = F_osc_BBC;
station_name = 'BBC Channel';
current_condition = 'Ideal';
receiver;
```

Listing 12: *Playing the demodulated signal's audio*

```
% Audio Playback
sound(demod_down, original_Fs);
pause((length(demod_down) / original_Fs) + 1);
```

Listening to the demodulated signal confirms that the original modulating message has been successfully recovered. Thanks to the high selectivity of the RF and IF stages, the output is completely free of adjacent channel interference. However, as the signal passes through multiple filters and processing stages, it inevitably experiences attenuation, resulting in a reduced overall amplitude. This loss in signal strength can be easily compensated for by integrating amplifiers after both the RF and IF stages.

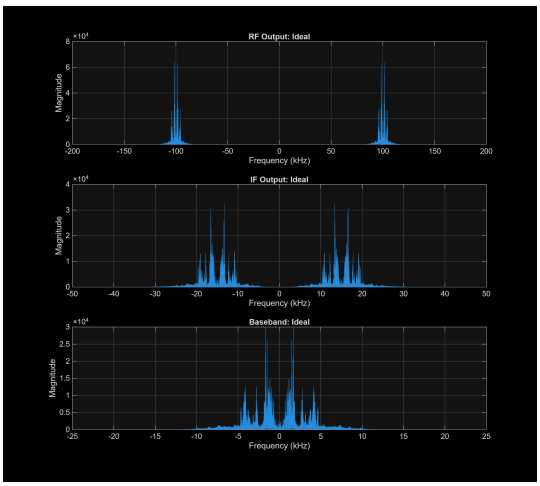

Figure 4.2: *The Quran Channel Spectrum.*

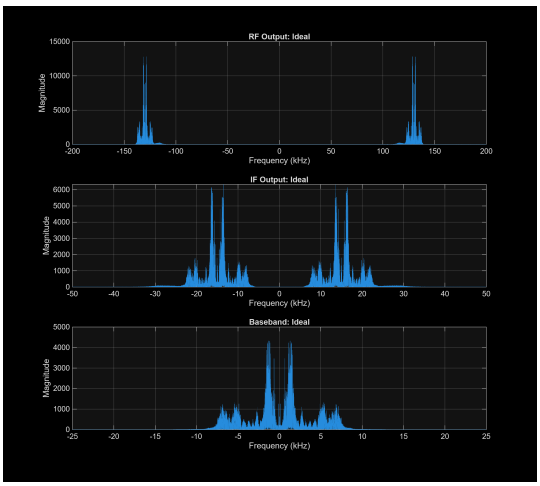

Figure 4.3: *The BBC Channel Spectrum.*

5

Effects of Removing the RF Band-Pass Filter (Q4)

Removing the RF Band-Pass Filter (BPF) leaves the receiver highly susceptible to severe interference from unwanted signals. Without this initial filtering stage, the mixer will down-convert both the desired station and an undesired "image" station into the same Intermediate Frequency (IF) band. This phenomenon is commonly known as the image frequency problem.

Note

In the case of high-side injection, the image frequency (F_{img}) is defined as:

$$F_{img} = F_{RF} + 2F_{IF} \tag{5.1}$$

We can implement this in our MATLAB code by passing the FDM directly to the receiver's IF stage, as demonstrated in Listing 13.

Listing 13: *Passing the FDM directly into the IF stage*

```
%% Q4 - NO RF Filter

% Quran Q4
current_RF_in = FDM; % Using raw FDM instead of filtered RF
current_F_osc = F_osc_Quran;
station_name = 'Quran Channel';
current_condition = 'Q4 (No BPF)';
receiver;

% BBC Q4
current_RF_in = FDM; % Using raw FDM instead of filtered RF
current_F_osc = F_osc_BBC;
station_name = 'BBC Channel';
current_condition = 'Q4 (No BPF)';
receiver;
```

As observed from the output spectrum in Figure 4.2, the Quran Channel (F_c = 100 kHz) suffers significantly from the image frequency problem. Given an intermediate frequency of F_{IF} = 15 kHz, substituting these values into Equation (5.1) yields an image frequency of F_{img} = 130 kHz. This calculated value coincides exactly with the carrier frequency of the BBC Channel, causing it to act as a direct source of interference.

Conversely, applying the same calculation to the BBC Channel (F_c = 130 kHz) results in an expected image frequency of F_{img} = 160 kHz. Since no active signal is transmitted at this frequency within our FDM system, the BBC Channel remains unaffected by image interference, as confirmed by the clean spectrum in Figure 4.3.

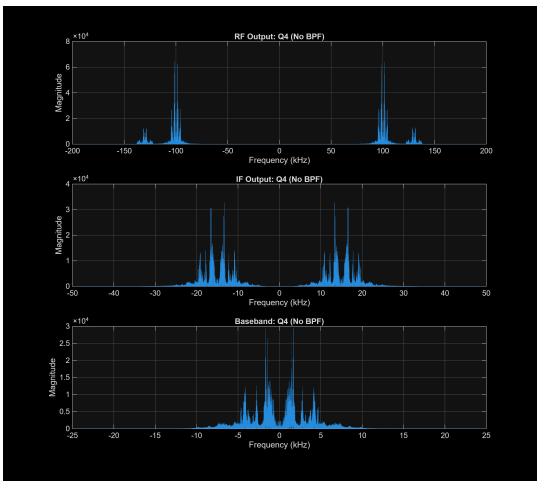

Figure 5.1: *The Quran Channel Spectrum (No RF BPF).*

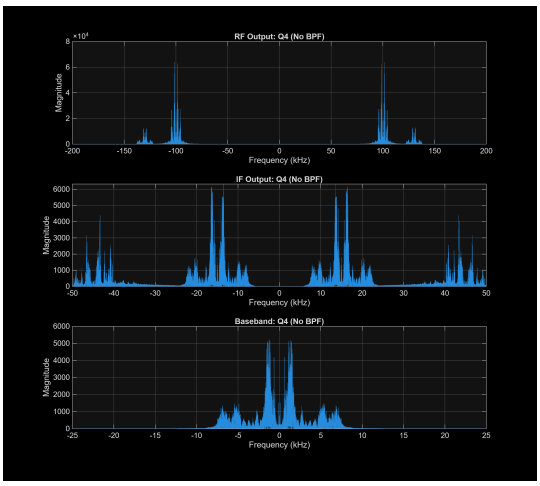

Figure 5.2: *The BBC Channel Spectrum(No RF BPF).*

As observed in Figure 5.1, the Quran Channel spectrum suffers from severe interference, with the BBC Channel's signal clearly visible alongside the desired frequency band. In contrast, the BBC Channel spectrum presented in Figure 5.2 remains virtually unaffected by the removal of the RF BPF, appearing identical to the filtered baseline previously established in Figure 4.3.

Warning

While an RF Band-Pass Filter (BPF) is beneficial but optional in a homodyne (zero-IF) receiver, it is absolutely essential in a heterodyne architecture to suppress image frequencies before they reach the mixing stage.

6
Adding Frequency offset (Q5)

In practice, analog components are never perfect and always introduce some offsets. The following section demonstrates the effects of both a small and a large offset on the system's performance. Fixing these hardware imperfections can be very expensive. Because of this, engineers must make a tradeoff based on the application: high-priority systems require spending more money to keep the offset as low as possible, while simpler applications can accept larger errors to save on costs.

6.1 Adding 0.1 kHz offset

We can start by adding a low offset $F_{offset} = 0.1\text{kHz}$

Listing 14: *Adding 0.1 kHz offset to the F_{LO}*

```
%% Q5 - MIXER OFFSET (0.1 kHz)
% Quran Q5 (0.1kHz)
current_RF_in = RF_output_Quran;
current_F_osc = F_osc_Quran + 0.1e3;
station_name = 'Quran Channel';
current_condition = 'Q5 (0.1kHz Offset)';
receiver;

% BBC Q5 (0.1kHz)
current_RF_in = RF_output_BBC;
current_F_osc = F_osc_BBC + 0.1e3;
station_name = 'BBC Channel';
current_condition = 'Q5 (0.1kHz Offset)';
receiver;
```

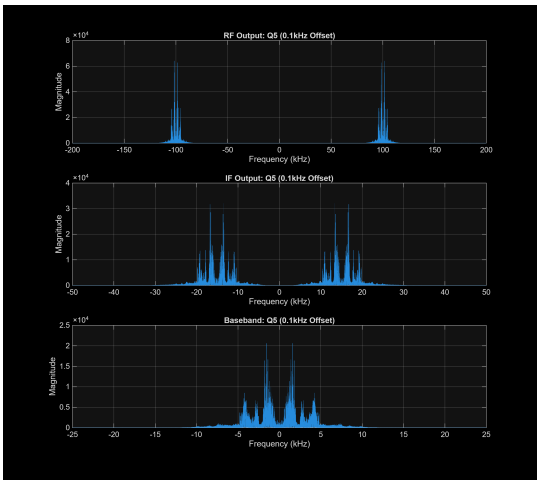

Figure 6.1: *The Quran Channel Spectrum (0.1 kHz Offset).*

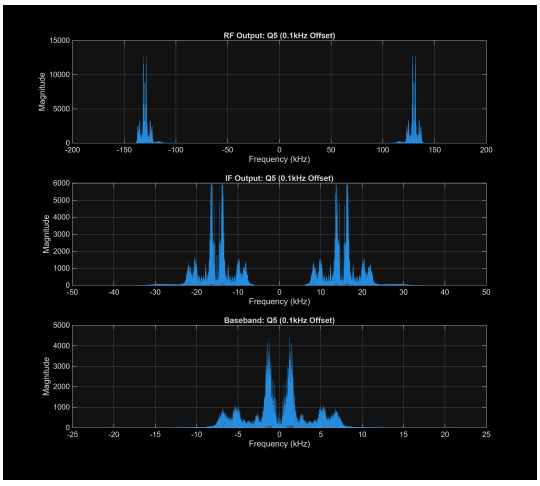

Figure 6.2: *The BBC Channel Spectrum(0.1 kHz Offset).*

By comparing the results in Figure 6.1 and Figure 6.2 to the original spectra, we can see that the signals are more distorted and show extra components in the center. When listening to the audio files using the code in Listing 12, the sound is still clear enough to understand, but it is noticeably more distorted than the original versions.

6.2 Adding 1 kHz offset

We will now try adding a higher offset that is F_{offset} = 1kHz

Listing 15: *Adding 1 kHz offset to the F_{LO}*

```
%% Q5 - MIXER OFFSET (1 kHz)
% Quran Q5 (1kHz)
current_RF_in = RF_output_Quran;
current_F_osc = F_osc_Quran + 1e3;
station_name = 'Quran Channel';
current_condition = 'Q5 (1kHz Offset)';
receiver;

% BBC Q5 (1kHz)
current_RF_in = RF_output_BBC;
current_F_osc = F_osc_BBC + 1e3;
station_name = 'BBC Channel';
current_condition = 'Q5 (1kHz Offset)';
receiver;
```

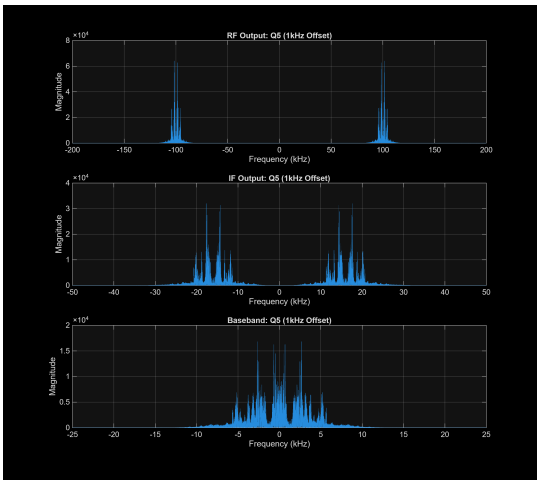

Figure 6.3: *The Quran Channel Spectrum (1 kHz Offset).*

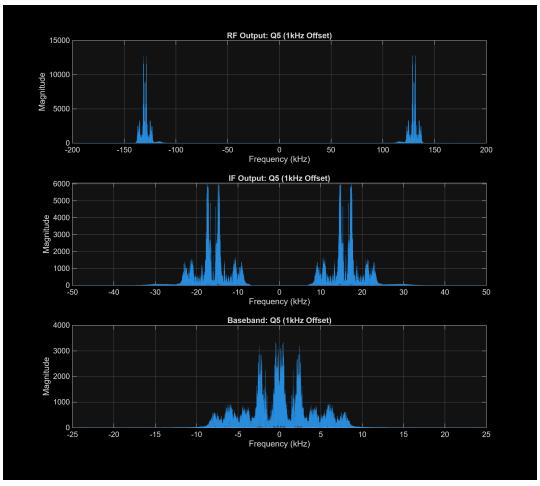

Figure 6.4: *The BBC Channel Spectrum(1 kHz Offset).*

When we look at the results in Figure 6.3 and Figure 6.4, the impact of the higher offset becomes much more obvious. The signals now show even more unwanted components and noise concentrated in the middle of the spectrum compared to our previous tests.

This technical distortion has a very clear effect on the final output. When we play the audio files, the sound quality has dropped so much that the original signal is now impossible to understand. Unlike the smaller offset where the voice was still clear, this larger offset creates so much interference that the audio is completely unhearable.

Table 6.1: *Comparison of results between different frequency offsets*

No Offset	0.1 kHz Offset	1 kHz Offset
The signal is perfect. There is no noise and the audio sounds exactly like the original.	The signal shows a small amount of distortion. The audio can be heard but it is not perfectly clear.	The signal is very distorted with many extra parts. The audio quality is gone and you cannot hear anything.

Appendices

A
Appendix

,

Listing 16: *Main Script — `AnalogCommProject.m`*

```
% main.m
% Authors : Youssef Hisham Ahmed, Youssef Mohammed Ibrahim
% Date : 4/4/2026
% Subject : Modulation schemes for various signals

clear; clc; close all;

%% Run Preprocessing and Modulator
preprocessing;
modulator;

%% Shared Filters Setup (IF & LPF)
F_IF = 15e3;

% IF Filtering Specs
IF_Filter_Order = 8;
IF_PassBand_Low = 5e3;
IF_PassBand_High = 30e3;
IF_Filter_Specs = fdesign.bandpass('N,F3dB1,F3dB2', IF_Filter_Order, IF_PassBand_Low,
↪  IF_PassBand_High, Fs1);
IF_filter = design(IF_Filter_Specs, 'butter');

% LPF Specs
LPF_Filter_Order = 8;
LPF_Cutoff = 10e3; % 10 kHz cutoff
LPF_Specs = fdesign.lowpass('N,F3dB', LPF_Filter_Order, LPF_Cutoff, Fs1);
LPF_filter = design(LPF_Specs, 'butter');

%% Channel Specific RF Filters & Oscillators
RF_Filter_Order = 26;

% Quran Channel RF Filter (100 kHz Center)
RF_PassBand_Low_Quran = 85e3;
RF_PassBand_High_Quran = 115e3;
RF_Filter_Specs_Quran = fdesign.bandpass('N,F3dB1,F3dB2', RF_Filter_Order,
↪  RF_PassBand_Low_Quran, RF_PassBand_High_Quran, Fs1);
```

```
RF_filter_Quran = design(RF_Filter_Specs_Quran, 'butter');
RF_output_Quran = filter(RF_filter_Quran, FDM);
F_osc_Quran = fc1 + F_IF;

% BBC Channel RF Filter (130 kHz Center)
RF_PassBand_Low_BBC = 115e3;
RF_PassBand_High_BBC = 145e3;
RF_Filter_Specs_BBC = fdesign.bandpass('N,F3dB1,F3dB2', RF_Filter_Order,
↪  RF_PassBand_Low_BBC, RF_PassBand_High_BBC, Fs1);
RF_filter_BBC = design(RF_Filter_Specs_BBC, 'butter');
RF_output_BBC = filter(RF_filter_BBC, FDM);
F_osc_BBC = fc2 + F_IF;

%% DEMODULATION

% Quran
current_RF_in = RF_output_Quran;
current_F_osc = F_osc_Quran;
station_name = 'Quran Channel';
current_condition = 'Ideal';
receiver;

% BBC
current_RF_in = RF_output_BBC;
current_F_osc = F_osc_BBC;
station_name = 'BBC Channel';
current_condition = 'Ideal';
receiver;

%% Q4 - NO RF Filter

% Quran Q4
current_RF_in = FDM; % Using raw FDM instead of filtered RF
current_F_osc = F_osc_Quran;
station_name = 'Quran Channel';
current_condition = 'Q4 (No BPF)';
receiver;

% BBC Q4
current_RF_in = FDM; % Using raw FDM instead of filtered RF
current_F_osc = F_osc_BBC;
station_name = 'BBC Channel';
current_condition = 'Q4 (No BPF)';
receiver;

%% Q5 - MIXER OFFSET (0.1 kHz)

% Quran Q5 (0.1kHz)
current_RF_in = RF_output_Quran;
current_F_osc = F_osc_Quran + 0.1e3;
station_name = 'Quran Channel';
current_condition = 'Q5 (0.1kHz Offset)';
```

```
receiver;

% BBC Q5 (0.1kHz)
current_RF_in = RF_output_BBC;
current_F_osc = F_osc_BBC + 0.1e3;
station_name = 'BBC Channel';
current_condition = 'Q5 (0.1kHz Offset)';
receiver;

%% Q5 - MIXER OFFSET (1 kHz)

% Quran Q5 (1kHz)
current_RF_in = RF_output_Quran;
current_F_osc = F_osc_Quran + 1e3;
station_name = 'Quran Channel';
current_condition = 'Q5 (1kHz Offset)';
receiver;

% BBC Q5 (1kHz)
current_RF_in = RF_output_BBC;
current_F_osc = F_osc_BBC + 1e3;
station_name = 'BBC Channel';
current_condition = 'Q5 (1kHz Offset)';
receiver;

disp('All processing complete!');
export_figures;
```

Listing 17: *Signal Pre-processing — `preprocessing.m`*

```
% preprocessing.m

%% Reading the files
[signal1_raw, Fs1] = audioread("Short_QuranPalestine.wav");
[signal2_raw, Fs2] = audioread("Short_BBCArabic2.wav");

%% Convert stereo to mono
if size(signal1_raw, 2) == 2, signal1 = signal1_raw(:,1) + signal1_raw(:,2); else, signal1 = signal1_raw; end
if size(signal2_raw, 2) == 2, signal2 = signal2_raw(:,1) + signal2_raw(:,2); else, signal2 = signal2_raw; end

disp(['Fs1 = ', num2str(Fs1), ' Hz']);
disp(['Fs2 = ', num2str(Fs2), ' Hz']);

%% Pad to equal length
len1 = length(signal1);
len2 = length(signal2);
max_len = max(len1, len2);

if len1 < max_len, signal1 = [signal1; zeros(max_len - len1, 1)]; end
```

```
if len2 < max_len, signal2 = [signal2; zeros(max_len - len2, 1)]; end

%% Plot Spectrum (Original Baseband)
N_plot1 = length(signal1);
Y1_shifted = fftshift(fft(signal1));
Y2_shifted = fftshift(fft(signal2));

magnitude1_shifted = abs(Y1_shifted);
magnitude2_shifted = abs(Y2_shifted);
freq_shifted = (-N_plot1/2 : N_plot1/2 - 1) * (Fs1 / N_plot1);

figure('Name', 'Original Baseband Spectra');
subplot(2,1,1);
plot(freq_shifted / 1000, magnitude1_shifted);
xlabel('Frequency (kHz)'); ylabel('Magnitude');
title('Spectrum of Signal 1 (Quran Channel)'); grid on; xlim([-25, 25]);

subplot(2,1,2);
plot(freq_shifted / 1000, magnitude2_shifted);
xlabel('Frequency (kHz)'); ylabel('Magnitude');
title('Spectrum of Signal 2 (BBC Channel)'); grid on; xlim([-25, 25]);
drawnow;

%% Upsampling
upsample_factor = 10;
signal1 = interp(signal1, upsample_factor);
signal2 = interp(signal2, upsample_factor);

original_Fs = Fs1;
Fs1 = Fs1 * upsample_factor;
Fs2 = Fs2 * upsample_factor;
disp(['New Fs1 = ', num2str(Fs1), ' Hz']);
disp(['New Fs2 = ', num2str(Fs2), ' Hz']);
```

Listing 18: *AM-DSB-SC Modulator — `modulator.m`*

```
% modulator.m
disp('--- Running Modulator ---');

%% MODULATION
N1 = length(signal1);
Ts = 1/Fs1;
t = (0:N1-1)' * Ts;

fc = 100e3;            % Base carrier frequency
delta_f = 30e3;        % Frequency spacing
fc1 = fc;              % Carrier for first signal (100 kHz)
fc2 = fc + delta_f;    % Carrier for second signal (130 kHz)

Quran_Channel_Modulated = signal1 .* cos(2 * pi * fc1 * t);
BBC_Channel_Modulated   = signal2 .* cos(2 * pi * fc2 * t);
```

```

FDM = Quran_Channel_Modulated + BBC_Channel_Modulated;

%% Plotting FDM
Y_FDM = fft(FDM);
Y_FDM_shifted = fftshift(Y_FDM);
magnitude_FDM = abs(Y_FDM_shifted);
freq_axis = (-N1/2 : N1/2 - 1) * (Fs1 / N1);
freq_plot = freq_axis;

figure('Name', 'FDM Spectrum');
plot(freq_axis / 1000, magnitude_FDM);
xlabel('Frequency (kHz)'); ylabel('Magnitude');
title('FDM Signal Spectrum'); grid on; xlim([-200, 200]);
hold on;
xline(fc1/1000, 'r--', '100 kHz');
xline(fc2/1000, 'g--', '130 kHz');
xline(-fc1/1000, 'r--', '-100 kHz');
xline(-fc2/1000, 'g--', '-130 kHz');
legend('FDM', 'Station 1 (Quran)', 'Station 2 (BBC)');
drawnow;
```

Listing 19: *Super-heterodyne Receiver Script — `receiver.m`*

```
% receiver.m

% Mixer Stage
Mixer_Output = current_RF_in .* cos(2 * pi * current_F_osc * t);

% IF Filtering Process
IF_Channel = filter(IF_filter, Mixer_Output);

% Return to Baseband & Low-Pass Filtering
baseband_mixed = IF_Channel .* cos(2 * pi * F_IF * t);
demodulated = filter(LPF_filter, baseband_mixed);
demodulated = demodulated - mean(demodulated); % Remove DC offset

% Downsample back & Normalize
demod_down = downsample(demodulated, upsample_factor);
demod_down = demod_down / max(abs(demod_down));

% Plotting
plot_spectra(current_RF_in, IF_Channel, demodulated, freq_plot, station_name,
↪  current_condition);

% Audio Playback
sound(demod_down, original_Fs);
pause((length(demod_down) / original_Fs) + 1);
```

Listing 20: *Spectrum Plotting Function — `plot_spectra.m`*

```
% plot_spectra.m
function plot_spectra(RF_sig, IF_sig, base_sig, freq_plot, channel_name, condition)

    figure('Name', sprintf('%s - %s', channel_name, condition));

    % RF Stage Plot
    subplot(3,1,1);
    plot(freq_plot / 1000, abs(fftshift(fft(RF_sig))));
    title(['RF Output: ', condition]);
    xlabel('Frequency (kHz)'); ylabel('Magnitude');
    grid on; xlim([-200, 200]);

    % IF Stage Plot
    subplot(3,1,2);
    plot(freq_plot / 1000, abs(fftshift(fft(IF_sig))));
    title(['IF Output: ', condition]);
    xlabel('Frequency (kHz)'); ylabel('Magnitude');
    grid on; xlim([-50, 50]);

    % Baseband Plot
    subplot(3,1,3);
    plot(freq_plot / 1000, abs(fftshift(fft(base_sig))));
    title(['Baseband: ', condition]);
    xlabel('Frequency (kHz)'); ylabel('Magnitude');
    grid on; xlim([-25, 25]);

    drawnow;
end
```